



Desenvolvimento Front-End

Prof. Elias A. da Silva
elias.asilva@sp.senac.br

CSS3 – Grid Layout

Em anexo a aula envio o código do estado atual, utilizem e façam os ajustes que acharem necessário.

O que vimos em aula passada foi referente ao conteúdo de Flexbox, não ignorem isso, pois é extremamente importante para o design de um site.

Hoje iremos adicionar a segunda ferramenta de layout: CSS Grid. E a diferença central é essa: Flexbox cuida de uma direção por vez, linha ou coluna. Grid cuida das duas ao mesmo tempo.

CSS3 – Grid Layout

Aulas
01–03

HTML5
Semantico
Formulários
Mídias

Concluída

Aulas
04–06

CSS3 + Flexbox
Tipografia
Variáveis

Concluída

Aula 07

CSS Grid
Layout

AGORA

Aula 08 - 09

Atividade
Portfólio

Atividade 1
Landing Page/PI

Aulas
10–20

JS · Publicação
Projeto Final

Portfólio atual:

HTML5 + CSS3 + Flexbox · Cores: #1F4E79 / #2E75B6 / #E05C00 · Fonte: Poppins · .cards-projetos e .form-grid já usam Flexbox — hoje migramos para Grid

CSS3 – Grid Layout vs Flexbox

FLEXBOX — 1 dimensao



elementos em fila (linha OU coluna), nao ambos

- ◆ header: logo + nav
- ◆ badges pill de habilidades
- ◆ botoes no hero
- ◆ footer centralizado
- ◆ form-grid (dois campos lado a lado)
- ◆ centralizar um unico item

V
S

CSS GRID — 2 dimensoes



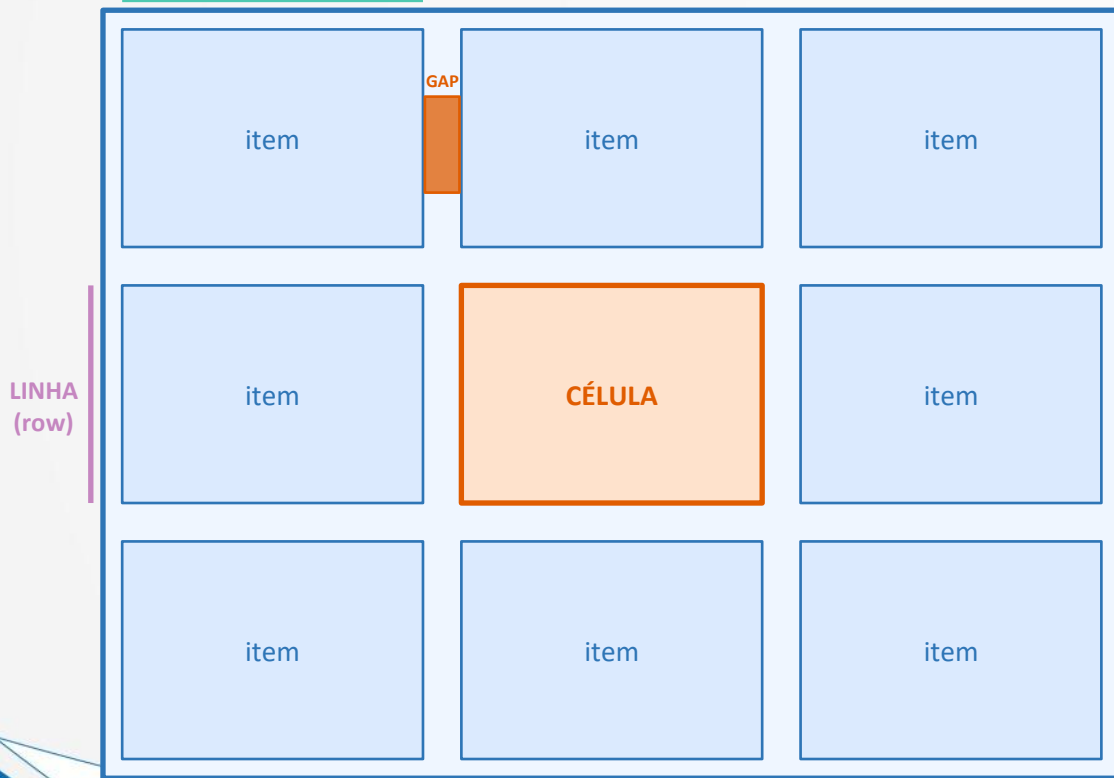
linhas E colunas ao mesmo tempo

- ◆ .cards-projetos (hoje — SNIPPET 02)
- ◆ layout hero: foto + texto (SNIPPET 03)
- ◆ layout de pagina: header/aside/main/footer
- ◆ dashboards e paineis
- ◆ galerias de imagens
- ◆ qualquer layout bidimensional

CSS3 – Anatomia do Grid

Entender o vocabulário e o primeiro passo para escrever Grid com confiança

COLUNA (column track)



CONTAINER

Elemento pai — recebe display: grid

ITEM

Cada filho direto do container

TRACK

Uma linha ou coluna inteira da grade

CÉLULA

Interseção de linha com coluna

AREA

Grupo nomeado de células adjacentes

GAP

Espaço entre as linhas e colunas

LINHA DE GRADE

Borda numerada separando os tracks

CONTAINER (display: grid)

CSS3 – GRID

Três propriedades para acionar qualquer grid

```
/* HTML */

<div class="grid">
  <div>Item 1</div>
  <div>Item 2</div>
  <div>Item 3</div>
</div>

/* CSS */

.grid {
  display: grid;
  grid-template-columns: 200px 1fr 2fr;
  grid-template-rows: auto 200px;
}
```

display: grid

Ativa o contexto de grid no container. Os filhos diretos tornam-se itens de grid automaticamente.

grid-template-columns

Define quantas colunas existem e qual o tamanho de cada uma. Aceita px, %, fr, auto e funções como repeat() e minmax().

grid-template-rows

Define a altura das linhas. Use auto para ajuste automático ao conteúdo ou valores fixos para controle preciso.

CSS3 – GRID

fr = fraction — distribui o espaço disponível de forma proporcional

1fr

1fr

2fr

grid-template-columns: 1fr 1fr 2fr → Total = 4 partes. 1fr = 25%. 2fr = 50%.

repeat(n, tamanho)

```
repeat(3, 1fr)  
→ 1fr 1fr 1fr
```

Evita repetição. Escreve o mesmo valor n vezes automaticamente.

minmax(min, max)

```
minmax(200px, 1fr)  
→ min:200px max:1fr
```

Define um intervalo. A coluna nunca fica menor que min nem maior que max.

auto-fill / auto-fit

```
repeat(auto-fill,  
  minmax(200px,1fr))  
→ grid responsivo automatico!
```

Cria quantas colunas couberem sem media query. Vamos usar isso no SNIPPET 02.

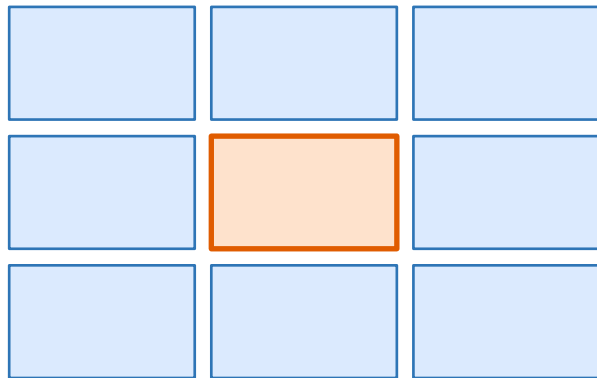
CSS3 – Grid Demonstração

Vocês irão usar o snippet 1 como base, a ideia é criar um novo html e um novo css, o que iremos fazer é experimentar antes de avançar para o portfólio.

Após digitado, verifiquem em tempo real no navegador, agora experimentem: troquem `repeat(3,1fr)` por `1fr 2fr 1fr`. O Grid divide o espaço em 4 partes, a do meio pega 2. Perceberam a unidade fr? Sem fazer conta. O navegador calcula por vocês.

Grid – Gap, Grid-column e Grid-row

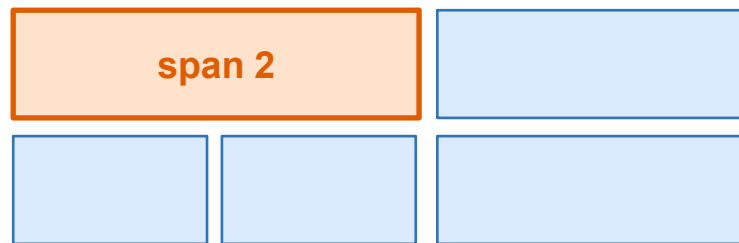
gap — espaço entre células



gap = row-gap + column-gap juntos

```
gap: 1rem;           /* linha e coluna */  
row-gap: 2rem;       /* so linhas */  
column-gap: 0.5rem;  /* so colunas */
```

grid-column e grid-row — span

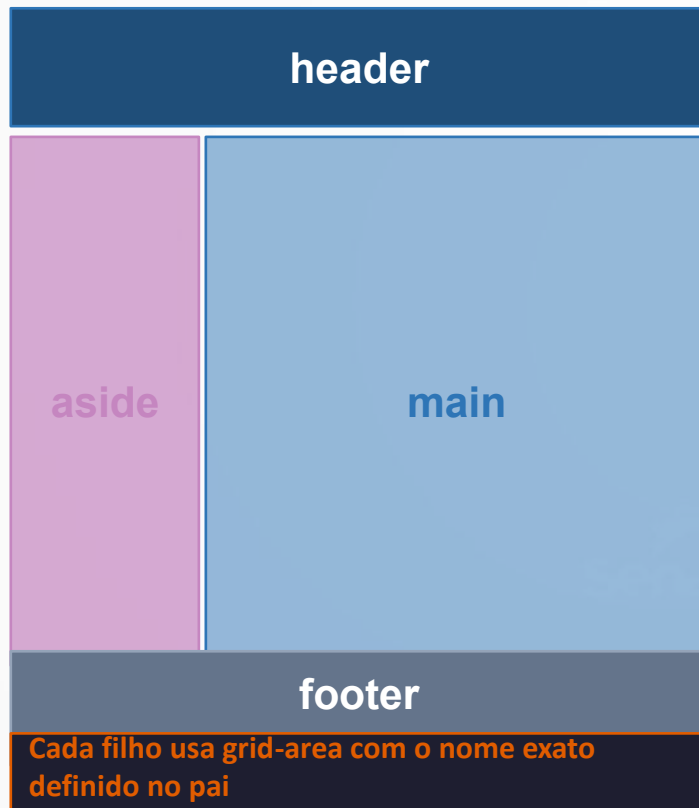


```
/* Ocupa N colunas */  
.destaque {  
  grid-column: span 2;  
  /* = grid-column: 1 / 3 */  
}  
  
.hero-full {  
  grid-column: 1 / -1; /* toda largura */  
  grid-row: span 2;  
}
```

CSS3 – Grid-Template-Areas

```
.pagina {  
  display: grid;  
  grid-template-columns: 1fr 3fr;  
  grid-template-rows:  
    80px 1fr 60px;  
  grid-template-areas:  
    "header header"  
    "aside main "  
    "footer footer";  
  gap: 1rem;  
  min-height: 100vh;  
}
```

```
header { grid-area: header; }  
aside { grid-area: aside; }  
main { grid-area: main; }  
footer { grid-area: footer; }
```



CSS3 – Grid – Prática

Acessem o portfólio, e acompanhem os snippets 2 e 3, implementem aqueles códigos incluído o bônus e façam as experimentações.

Abram o portfolio no navegador. Redimensionem a janela. Os cards se reorganizam sozinhos. Três colunas em tela larga, dois em media, um no celular. Sem escrever uma media query.

No bônus o card em destaque irá ocupar duas colunas ao final.



CSS3 – Grid – Prática

No Snippet 3, iremos atingir o estado onde a foto ficou a esquerda, titulo e descrição a direita. Isso foi grid-template-areas aplicado num elemento real do portfolio de vocês.

Notem: o HTML não mudou. Os seletores .hero h2 e .hero p são seletores descendentes, já funcionavam, agora recebem grid-area.

Para o final desta aula, quero que customizem o portfólio de vocês com dados, e deixem funcional, pesquisem sobre Media Query e Responsividade, sugestão: usem o W3Schools, implementem algo disso no portfólio de vocês, e realizem a entrega do portfólio.

CSS – O que aplicamos hoje

DevTools — Grid Inspector

- 1 Abra o DevTools (F12) e localize o container
- 2 Clique no badge de grade ao lado do elemento
- 3 O navegador sobrepoe linhas e areas coloridas
- 4 Aba Layout mostra todas as grades da pagina
- 5 Numeros nas bordas = indices para grid-column/row

O que aprendemos hoje

- ✓ Grid vs Flexbox — cada ferramenta no seu contexto
- ✓ Anatomia: container, item, track, celula, area, gap
- ✓ display: grid, template-columns, template-rows
- ✓ Unidade fr, repeat(), minmax(), auto-fill
- ✓ gap, grid-column / row, span
- ✓ grid-template-areas — layout por nomes
- ✓ Portfolio: cards responsivos + hero reorganizado

Proxima aula — **Aula 08: Responsividade** | media queries, mobile-first, viewport

CSS3 – FlexBox – Grid – Media Query

Para o final desta aula, quero que customizem o portfólio de vocês com dados, e deixem funcional, pesquisem sobre Media Query e Responsividade, sugestão: usem o W3Schools, implementem algo disso no portfólio de vocês, e realizem a entrega do portfólio. Essa entrega é para intuito de avaliação, e em paralelo está a outra atividade programada para entrega na aula seguinte.